

# pa7uiclf

## Ciência da Computação

Análise de Algoritmos com Listas Encadeadas  
Circulares, ordenações e recursividade.

29/08/2025

# Estrutura de dados - Listas

- **Revisão listas simples e duplamente encadeadas**
- **Listas encadeadas circulares**
- **Ordenação de Listas encadeadas**
- **Recursividade**
- **Exercícios**

# Estrutura de dados - Listas circulares

## Objetivos da Aula:

- Apresentar o conceito e as diferenças entre uma lista encadeada circular e uma lista encadeada simples.
- Demonstrar as operações básicas de inserção, remoção e travessia em uma lista encadeada circular.
- Discutir a complexidade e os métodos para ordenar listas encadeadas.
- Capacitar, implementar e manipular listas encadeadas circulares e ordenadas em código.
- Recursividade e onde utilizar

# Estrutura de dados - Listas

- **Revisão: Listas simplesmente e duplamente encadeadas**

# Estrutura de dados - Listas



## Revisão Rápida: Listas Encadeadas Simples

- O que é uma lista encadeada? (Estrutura de dados com nós, cada um contendo dados e um ponteiro para o próximo nó.)
- Qual é a principal diferença entre uma lista encadeada e um array? (Alocação dinâmica, sem necessidade de alocar memória contígua.)
- Onde termina uma lista encadeada simples? (O último nó aponta para **NULL** ou **None**.)

# Estrutura de dados - Listas

Uma lista duplamente encadeada é uma variação da lista encadeada, onde **cada nó tem um ponteiro para o próximo nó (next) e outro para o nó anterior (prev)**.

## Estrutura e Vantagens

- **Nó:** Cada elemento armazena os dados e os dois ponteiros, para o próximo e para o anterior.
- **Navegação:** A grande vantagem é poder percorrer a lista em **ambas as direções** (para frente e para trás).
- **Eficiência:** Operações como a **remoção de um nó** tornam-se mais eficientes, pois não é preciso percorrer a lista inteira para encontrar o nó anterior.

## Operações-Chave

- **Inserção:** Envolve a atualização de múltiplos ponteiros (**next** e **prev**) para manter a conexão bidirecional.
- **Remoção:** É mais simples do que na lista encadeada simples, pois o acesso ao nó anterior é direto.

# Estrutura de dados - Listas

| Característica   | Lista Simplesmente Encadeada                 | Lista Duplamente Encadeada                 |
|------------------|----------------------------------------------|--------------------------------------------|
| <b>Ponteiros</b> | Apenas <code>next</code>                     | <code>next</code> e <code>prev</code>      |
| <b>Navegação</b> | Apenas para frente                           | Para frente e para trás                    |
| <b>Remoção</b>   | Mais lenta (precisa encontrar o nó anterior) | Mais rápida (acesso direto ao nó anterior) |
| <b>Memória</b>   | Menor consumo                                | Maior consumo (2 ponteiros por nó)         |

# Estrutura de dados - Listas circulares

## Introdução às Listas Encadeadas Circulares

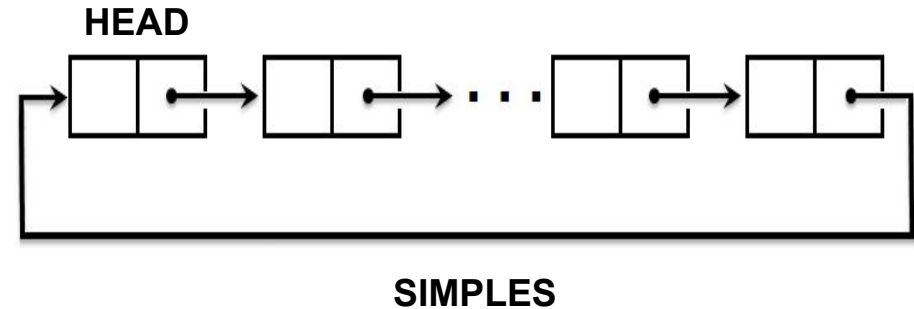
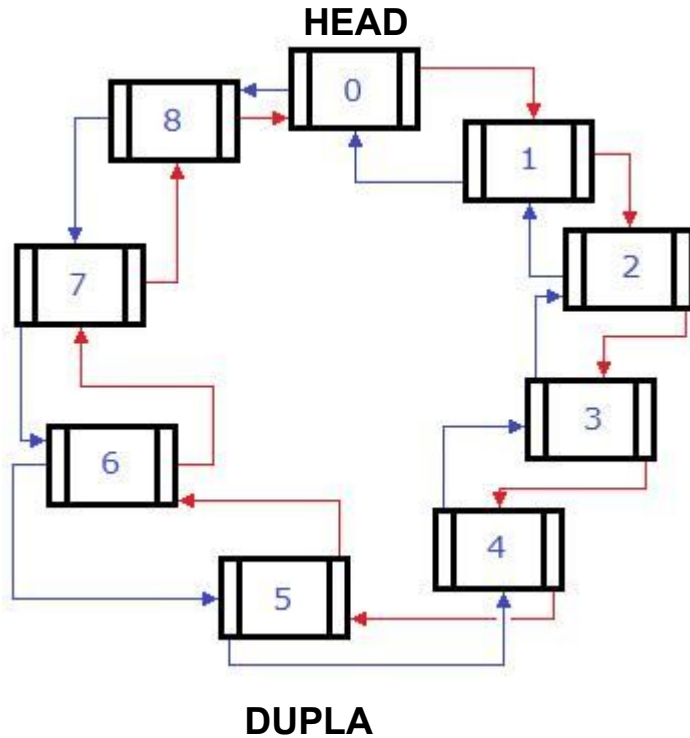
O conceito de forma simples: em vez de o último nó apontar para **NULL**, ele aponta de volta para o primeiro nó (a cabeça da lista). Isso cria um "círculo".

**Exemplo Prático:** Use uma analogia, como a de uma fila de pessoas de mãos dadas, onde a última pessoa segura a mão da primeira.

### Principais Características:

- Não há um "fim" definido. A travessia da lista é contínua.
- É possível acessar qualquer nó a partir de qualquer outro nó, simplesmente percorrendo a lista.

# Estrutura de dados - Lista Encadeada Circular



# Estrutura de dados - Lista Encadeada Circular

## Operações em Listas Circulares

- **Travessia (Percorrer a Lista):**
  - O loop de travessia precisa de uma condição de parada diferente. Em vez de verificar se o nó atual é **NULL**, o loop deve parar quando o nó atual for novamente o nó inicial (a cabeça).

**Exemplo:** <https://trinket.io/java/37dbc9790ec1> - Simples

**Exemplo:** <https://trinket.io/java/62000ff73723> - Dupla

- **Ordenação de Listas Encadeadas**

# Estrutura de dados - Ordenação de Listas Encadeadas



## O Desafio da Ordenação

Como podemos ordenar uma lista encadeada? Podemos usar os mesmos algoritmos de ordenação que usamos para arrays?

- Os algoritmos como **Merge Sort** e **Quick Sort** funcionam bem, mas com algumas adaptações, pois não há acesso aleatório aos elementos (não se pode usar `array[i]`).

## Merge Sort

O Merge Sort é um algoritmo de divisão e conquista que funciona dividindo a lista recursivamente em duas metades até que cada sub-lista tenha apenas um elemento. Depois, ele junta (merge) essas sub-listas ordenadamente.

- **Divisão:** A lista é dividida ao meio, de forma eficiente, usando a técnica dos ponteiros rápido e lento. Essa operação é muito rápida em uma lista encadeada. Conceito: **fast and slow pointer**
- **Conquista:** As duas metades são ordenadas recursivamente.
- **Combinação:** As duas sub-listas ordenadas são mescladas em uma única lista. Isso é feito simplesmente ajustando os ponteiros **next** dos nós, sem precisar mover os dados.

O **Merge Sort** é considerado a melhor escolha para listas encadeadas porque a divisão e a junção são operações naturais e eficientes para essa estrutura de dados, pois não exigem acesso aleatório (acesso a um elemento por índice).

## Quick Sort

O Quick Sort também é um algoritmo de divisão e conquista, mas sua lógica é diferente. Ele escolhe um elemento como **pivô** e particiona a lista em duas partes: uma com elementos menores que o pivô e outra com elementos maiores.

- **Particionamento:** A lista é percorrida e os nós são movidos para duas novas listas: uma de menores e outra de maiores.
- **Conquista:** O Quick Sort é chamado recursivamente nas duas novas sub-listas (a dos menores e a dos maiores).
- **Combinação:** As listas ordenadas são unidas com o pivô no meio.

O **Quick Sort** é mais complexo em listas encadeadas porque a etapa de particionamento, que é muito rápida em arrays, se torna mais lenta e complicada. A implementação exige criar sub-listas ou manipular muitos ponteiros, o que é propenso a erros.

# Estrutura de dados - Ordenação de Listas Encadeadas

## Diferenças Principais

| Característica                | Merge Sort                                                                    | Quick Sort                                                                                                       |
|-------------------------------|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Abordagem</b>              | Divide até a menor parte e depois junta.                                      | Particiona em torno de um pivô e depois ordena as subpartes.                                                     |
| <b>Eficiência para Listas</b> | Mais eficiente e natural. A divisão e junção são operações rápidas e seguras. | Menos eficiente e mais complexo. A etapa de particionamento é custosa e exige manipulação complexa de ponteiros. |
| <b>Estabilidade</b>           | É um algoritmo <b>estável</b> (mantém a ordem relativa de elementos iguais).  | É um algoritmo <b>instável</b> (não garante a ordem de elementos iguais).                                        |
| <b>Custo de Memória</b>       | Requer alocação de memória extra para as sub-listas temporárias na junção.    | Requer menos memória extra para o particionamento, mas a recursão usa espaço na pilha.                           |

# Estrutura de dados - Ordenação de Listas Encadeadas



- **Recursividade**

A **recursividade** é uma técnica de programação onde uma função chama a si mesma para resolver um problema. É como resolver um grande problema quebrando-o em versões menores e idênticas de si mesmo, até chegar a um caso tão simples que a resposta é óbvia.

## Como Funciona a Recursividade

Uma função recursiva precisa de duas partes essenciais:

1. **Caso Base:** É a condição de parada. É o cenário mais simples do problema, onde a função não se chama mais, retornando um valor diretamente. Sem o caso base, a função entraria em um loop infinito.
2. **Passo Recursivo:** É a parte onde a função chama a si mesma com uma versão simplificada do problema original. A cada chamada, o problema se aproxima do caso base.

## Qualidades e Vantagens da Recursividade

- **Elegância e Clareza do Código:** Para problemas que têm uma estrutura inerentemente recursiva (como a travessia de árvores, o cálculo de fatoriais ou a busca em grafos), a solução recursiva costuma ser mais curta, elegante e fácil de entender do que uma solução iterativa (com `for` ou `while`).
- **Resolução de Problemas Complexos:** A recursividade é ideal para resolver problemas complexos que podem ser divididos em subproblemas idênticos, como os algoritmos de ordenação **Merge Sort** e **Quick Sort**. Ela reflete a forma como pensamos em certos problemas, tornando a lógica mais intuitiva.
- **Menos Variáveis:** A recursividade muitas vezes elimina a necessidade de variáveis de controle complexas, como contadores e flags que seriam necessários em loops iterativos.

## Desvantagens e Cuidados

- Apesar de suas qualidades, a recursividade não é a melhor solução para todos os problemas. Ela pode consumir mais memória, pois cada chamada de função é empilhada na memória do sistema. Em casos de recursão profunda, isso pode levar a erros de "Stack Overflow" (estouro de pilha). Por isso, é crucial sempre definir um caso base que garanta o término da execução.

## Quando a Recursividade Vale a Pena?

Apesar dos desafios, a recursividade é a melhor escolha para problemas que têm uma estrutura de dados ou lógica recursiva, como:

- **Algoritmos de Divisão e Conquista:** Merge Sort, Quick Sort e busca binária.
- **Estruturas de Dados Recursivas:** Travessia de árvores (pré-ordem, em-ordem, pós-ordem) e grafos.
- **Problemas com Lógica Recursiva Pura:** Cálculo de fatoriais, sequências de Fibonacci (embora ineficiente, é um bom exemplo didático).

# Estrutura de dados - Recursividade

- Listagem de Arquivos ([java.io.File](#))

```
public void listarArquivos(File diretorio) {  
    File[] arquivos = diretorio.listFiles();  
    if (arquivos != null) {  
        for (File f : arquivos) {  
            if (f.isDirectory()) {  
                // Chamada recursiva para subdiretório  
                listarArquivos(f);  
            } else {  
                // Processa o arquivo  
                System.out.println(f.getName());  
            }  
        }  
    }  
}
```

# Estrutura de dados - Recursividade

- Frameworks de Persistência (JPA/Hibernate)

Em frameworks de mapeamento objeto-relacional como o **Hibernate**, os desenvolvedores definem relações entre entidades com anotações como `@OneToMany` e `@ManyToOne`. Quando você carrega uma entidade principal (por exemplo, um `Departamento`), e ela tem uma coleção de entidades relacionadas (`Empregados`) com uma estratégia de busca (`fetch`) do tipo `EAGER`, o Hibernate precisa:

1. Consultar o `Departamento`.
2. Para cada `Departamento`, consultar e carregar seus `Empregados`.
3. Se um `Empregado` tiver uma outra coleção relacionada (por exemplo, `Projetos`), ele precisa carregar esses `Projetos` também.

Esse processo de carregar um **grafo de objetos** a partir do banco de dados é um problema de travessia que, internamente, é frequentemente resolvido de forma recursiva. O desenvolvedor simplesmente escreve o código para obter a entidade `Departamento` e o framework faz todo o trabalho de forma transparente.

# Estrutura de dados - Listas

- **Exercícios (Arquivo Algo-Merge.txt)**

# Estrutura de dados - Ordenação de Listas Encadeadas



**Praticar!**

<https://trinket.io/java/3877974e5981> - Quicksort

<https://trinket.io/java/5a0d07bd7a7b> - Merge

<https://trinket.io/java/3bea14a5d05a> - Merge com Lista Circular

# Simulado

